Emergency_Response_SOP

Technical Standard

保险公司 技术开发部
内部文档 · 仅供授权人员查阅

Contents

1. Overview
   1.1 Purpose
   1.2 Scope
2. Incident Severity Levels
   2.1 Classification
   2.2 Financial Loss Incidents
3. Incident Response Organization
   3.1 Roles
   3.2 Escalation Chain
4. Incident Response SOP
   4.1 Detection & Response
   4.2 Investigation & Diagnosis
   4.3 Mitigation & Recovery
   4.4 Recovery & Confirmation
5. Runbooks
   5.1 Core Service Down
   5.2 Data Inconsistency
   5.3 Database Failure
6. Postmortem
   6.1 Principles
   6.2 Postmortem Template
Basic Info
Timeline
Root Cause Analysis
   Direct Cause
   Root Cause
Response Assessment
Action Items
7. Appendix
   7.1 Emergency Contacts

Emergency Response Plan & SOP

> Reference: Google SRE Incident Response Practices, Alibaba Cloud Fault Management SOP, Tencent Emergency Response Standards, ITIL Incident Management

1. Overview

1.1 Purpose

Establish standardized incident response mechanisms and handling procedures to ensure rapid detection, efficient recovery, and continuous improvement when core service incidents occur, minimizing business impact.

1.2 Scope

Applies to all production environment incidents and emergencies for systems managed by the Technology Development Department.

2. Incident Severity Levels

2.1 Classification

Level	Definition	Example	Response Time	Escalation Time
S0 (Catastrophic)	Core system completely unavailable, or financial loss	Core outage, accounting error, data loss	Immediate	10min
S1 (Severe)	Core module severely impaired, broad impact	Policy issuance unavailable, payment failure, widespread slowness	5min	30min
S2 (Moderate)	Non-core functions impaired, workaround exists	Report export failure, partial channel issues	30min	2h
S3 (Minor)	Minor anomaly, no business impact	Intermittent single API timeout, false alert	24h	---

2.2 Financial Loss Incidents

Any incident involving financial loss is automatically classified as S0 and requires:

1. Immediate loss prevention (stop transactions / degrade feature)
2. Notify finance department
3. Preserve all logs and database snapshots
4. Legal department involvement for loss assessment and compliance risk

3. Incident Response Organization

3.1 Roles

Role	Person	Responsibility
Incident Commander (IC)	On-call Tech Lead	Overall incident management, resource allocation, external communication

Role	Person	Responsibility
First Responder	On-call Engineer	Initial investigation, execute mitigation actions
Technical Expert	Module owner	Deep investigation, fix solution design
Communications Lead	PM/PMO	Progress updates to business and management
Scribe	On-call staff/intern	Record incident timeline and actions

3.2 Escalation Chain

```
First responder detects incident
↓
IC confirms severity level
↓
S0/S1 > Immediate incident group (tech + business + management)
S2 > Notify tech lead and module team
S3 > Log ticket, handle during business hours
↓
Recovery > Postmortem
```

---

4. Incident Response SOP

4.1 Detection & Response

1. Alert triggered / User feedback / On-call inspection discovers incident
2. Confirm: Is it real? Impact scope? Severity level?
3. IC takes command, establish incident communication group
4. Initial announcement within 5 minutes

Initial Announcement Template:

```
[INCIDENT] [System] [Severity]
Symptom: [Description]
Start Time: [Time]
Impact: [Affected functions/users]
Status: [Investigating/Resolving/Resolved]
Estimated Recovery: [Time]
IC: [Name]
```

4.2 Investigation & Diagnosis

1. Check monitoring dashboards (Grafana / Kibana)
2. Check recent changes (caused by a change?)
3. Check application logs (ERROR levels / stack traces)
4. Check database status (connections / slow queries / replication lag)
5. Check infrastructure (CPU / memory / disk / network)
6. Check external dependencies (third-party APIs / middleware)

Investigation Checklist:

```
□ Any recent changes? (check change management system)
□ Database slow queries or deadlocks? (check slow query log)
□ Sudden traffic spike? (check QPS monitoring)
□ External dependencies healthy? (check health endpoints)
□ Scheduled tasks running? (check scheduler)
□ Certificates/keys expired? (check certificate validity)
```

4.3 Mitigation & Recovery

Incident Type	Action	Damage Control
Application down	Restart / Rollback	Route traffic to standby
DB deadlock	Kill blocking session / Optimize SQL	Primary-standby switch
Slow query resource exhaustion	Kill slow query / Add index	Rate limiting / Degrade
Middleware failure	Restart / Switch cluster	Degrade to local cache
Network failure	Switch DNS / Switch AZ	CDN / Multi-Active
Data error	Restore from backup / Data correction	Suspend related transactions
Security attack	Block IP / Enable WAF	Degrade / Switch to standby

Execution Principles:

- **Stop loss first**: Restore service first, investigate root cause later
- **Rollback first**: If caused by a change, prefer rollback over online fix
- **Four-eyes principle**: High-risk operations require dual confirmation
- **Record every step**: Scribe records all operations and timestamps

4.4 Recovery & Confirmation

```
1. Execute recovery action
2. Verify service health (/health + core API smoke test)
3. Confirm business metrics recovered (policy issuance/payment success rate normal)
4. Monitor for 30 minutes (no abnormal fluctuation)
5. Clear alerts, send recovery notice
6. Close incident group (move to postmortem phase)
```

---

5. Runbooks

5.1 Core Service Down

```
Applies to: Policy Service / Underwriting Service unresponsive
Steps:
1. Check service process status (ps / kubectl get pods)
2. Check recent logs (last 5 min ERROR and stack traces)
3. If OOM: increase memory config, restart instance
4. If deadlock/GC issue: capture Thread Dump and Heap Dump
5. If network issue: check service discovery (Nacos/Eureka) and network policy
6. If code defect: rollback to previous stable version
7. After recovery: gradually restore traffic, observe 30 min
```

5.2 Data Inconsistency

```
Applies to: Core system vs fund platform reconciliation mismatch, policy status inconsistency
Steps:
1. First, verify data difference, confirm impact scope
2. Assess whether to suspend related transactions
3. If accounting mismatch: switch to backup reconciliation flow, preserve source documents
4. If status mismatch: fix via compensation transaction or data correction script
5. Record all manual corrections (who, what, when, why)
6. Full reconciliation verification after correction
7. Postmortem: Add reconciliation monitoring and prevention mechanisms
```

5.3 Database Failure

```
Applies to: Connection pool full, replication lag excessive, disk space full
Replication Lag:
```

```
1. Check replica process status (SHOW SLAVE STATUS)
2. If lag increasing, check large transactions or slow queries
3. Emergency: skip error transaction (use with caution) or rebuild replica
4. Long-term: optimize large transactions, upgrade replica config
Connection Pool Full:
1. Check active and waiting connections
2. Kill idle connections or long-running uncommitted transactions
3. Check for connection leaks (connection pool monitoring)
4. Emergency increase connection pool limit
Disk Space Full:
1. Clean archive logs (binlog / slow log)
2. Clean temporary tablespace
3. Emergency disk expansion
4. Review cleanup policies
```

6. Postmortem

6.1 Principles

Principle	Description
Blameless	Goal is to improve the system, not assign blame
Root Cause Focus	Find the root cause, not just symptoms
Systemic	Individual errors often reveal process/tool/training gaps
Actionable	Every action item must have owner and deadline

6.2 Postmortem Template

```
# Postmortem - [Incident ID]
## Basic Info
- Title: [Title]
- Severity: [S0/S1/S2/S3]
- Duration: [Start] - [End]
- Total Time: [X] minutes
- Impact: [Affected systems/functions/users]
- Financial Loss: [If applicable]
## Timeline
| Time | Event | Person |
| --- | --- | --- |
| [T0] | Incident occurs | System |
| [T+5min] | Alert triggered | System |
| [T+10min] | IC confirms S1 | [Name] |
| [T+20min] | Root cause found: ... | [Name] |
| [T+30min] | Rollback executed | [Name] |
| [T+45min] | Service recovered | [Name] |
## Root Cause Analysis
### Direct Cause
[Trigger condition that caused the incident]
### Root Cause
[List of contributing factors]
## Response Assessment
| Aspect | Rating | Improvement |
| --- | --- | --- |
| Detection speed | Fast/Medium/Slow | [Suggestion] |
| Diagnosis efficiency | High/Medium/Low | [Suggestion] |
| Mitigation effectiveness | Effective/Ineffective | [Suggestion] |
| Communication quality | Good/Average/Poor | [Suggestion] |
## Action Items
| No | Action | Owner | Due Date |
| --- | --- | --- | --- |
| 1 | [Action] | [Name] | [Date] |
```

```
| 2 | [Action] | [Name] | [Date] |
```

7. Appendix

7.1 Emergency Contacts

Role	Person	Phone	Backup
IC (This week)	[Name]	[Phone]	IM
On-call DBA	[Name]	[Phone]	IM
Security Lead	[Name]	[Phone]	IM

7.2 Reference Documents

- Google SRE - Incident Response
- ITIL Incident Management
- *Logging & Monitoring Standard*
- *Release & Change Management Standard*

7.3 Quick Command Reference

```
# Check application logs
tail -100f /var/log/app/error.log
# Check process status
systemctl status policy-service
kubectl get pods -n insurance
# Database operations
mysql -h host -u user -p -e "SHOW PROCESSLIST;"
mysql -h host -u user -p -e "SHOW SLAVE STATUS\G"
# Check disk
df -h
du -sh /var/log/
# Network diagnostics
curl -I http://service-name:8080/health
telnet host port
```